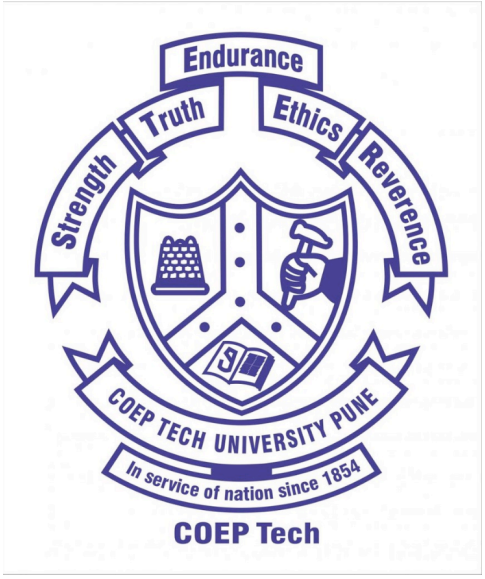
Road Accident Hotspot Analysis and Severity Prediction System



Department : Computer Science and Engineering

GROUP MEMBERS

Siddhant Hemagire – 612415067

Krishiv Nair – 612415087

Makarand Kulkarni – 612415092

1. Problem Statement

Road accidents cause heavy loss of life and property. Accident records are often scattered across paper registers and flat spreadsheets. Location, road, weather and cause details are repeated in every row. This leads to redundancy, inconsistent spellings ("Rain" vs "rainy") and update anomalies. It also makes simple questions hard to answer: which junctions are most dangerous, what conditions make a crash fatal, and how crash volume changes over time.

This project designs and implements a normalized relational database, with a web application on top, for the **Road Accident Hotspot Analysis and Severity Prediction System**. It stores accidents together with their location, road, weather, cause, vehicles and victims. It ranks hazardous locations (hotspots) and estimates the likely severity of a crash (Minor, Serious or Fatal) from historical data.

2. Objectives

- Design a conceptual ER model for road accidents and map it to a relational schema in **Third Normal Form (3NF)**.
- Remove redundancy and anomalies by moving repeated facts into master and lookup tables, linked by foreign keys.
- Enforce data integrity with primary, foreign and unique keys, CHECK constraints, NOT NULL and referential actions.
- Identify accident **hotspots** with a composite risk score computed in a SQL view.
- Analyse accidents by cause, weather, month and time of day using aggregate queries.
- **Predict severity** from road type, weather, cause and time band using a MySQL stored procedure over historical records.
- Guarantee **ACID** behaviour when one incident is recorded across several tables (accident, vehicles, victims).
- Provide a usable three-tier application (React front end, Node/Express API, MySQL database).

3. Scope

In scope

- A MySQL 8 database `accident_db` with 9 relations, an analytical view `v_hotspot_ranking` and the stored procedure `sp_predict_severity`.
- A sample dataset of 10 locations, 8 roads, 5 weather conditions, 6 causes, 5 vehicle types, 840 accidents, 1,310 accident-vehicle links and 1,131 victims, generated by a script covering all 360 road-type, weather, cause and time-band combinations.
- A REST API with five endpoints: hotspots, statistics, lookups, predict and add accident.
- A web interface with four modules: Hotspots, Predictor, Analytics and Add Accident.

Out of scope

- Live GPS or sensor feeds, map rendering and integration with police or hospital systems.
- User authentication and role-based access.
- Machine-learning models. Prediction is frequency-based (empirical probability), not a trained classifier.
- Vehicle-level details such as registration number, driver licence or insurance, since only vehicle *types* are stored.

4. Functional Requirements

ID	Requirement
FR1	Store master data: locations (name, district, latitude, longitude) and roads (name, type, lanes, surface, speed limit).
FR2	Maintain lookup catalogs for weather conditions, severity levels, accident causes and vehicle types.
FR3	Record an accident with date, time, description, location, road, weather, overall severity and cause.
FR4	Record one or more vehicle types involved in each accident (many-to-many).
FR5	Record zero or more victims per accident with age, gender, person type (Driver, Passenger or Pedestrian), hospitalization status and injury severity.
FR6	Add a new accident with its vehicles and victims as a single atomic transaction. If any insert fails, everything is rolled back.
FR7	Rank locations by total accidents, fatal accidents and risk score (Minor = 1, Serious = 2, Fatal = 3, summed per location).
FR8	Provide analytics: accidents and fatalities by cause, severity mix by weather, monthly trend and time-of-day distribution.
FR9	Predict severity for a chosen road type, weather, cause and time band, showing the probability of each level. If no matching history exists, fall back to the overall distribution.
FR10	Validate input and enforce constraints (coordinate ranges, allowed road types, genders and person types, positive lanes and speed limit, age 0 to 125).
FR11	Prevent deletion of a master or lookup row that is still referenced. Deleting an accident also deletes its vehicle links and victims.

Non-functional requirements: parameterised SQL only (no SQL injection), connection pooling, indexes on frequently filtered columns, and 3NF to avoid redundancy.

5. ER Diagram

Entities and key attributes. Keys are underlined in the diagram.

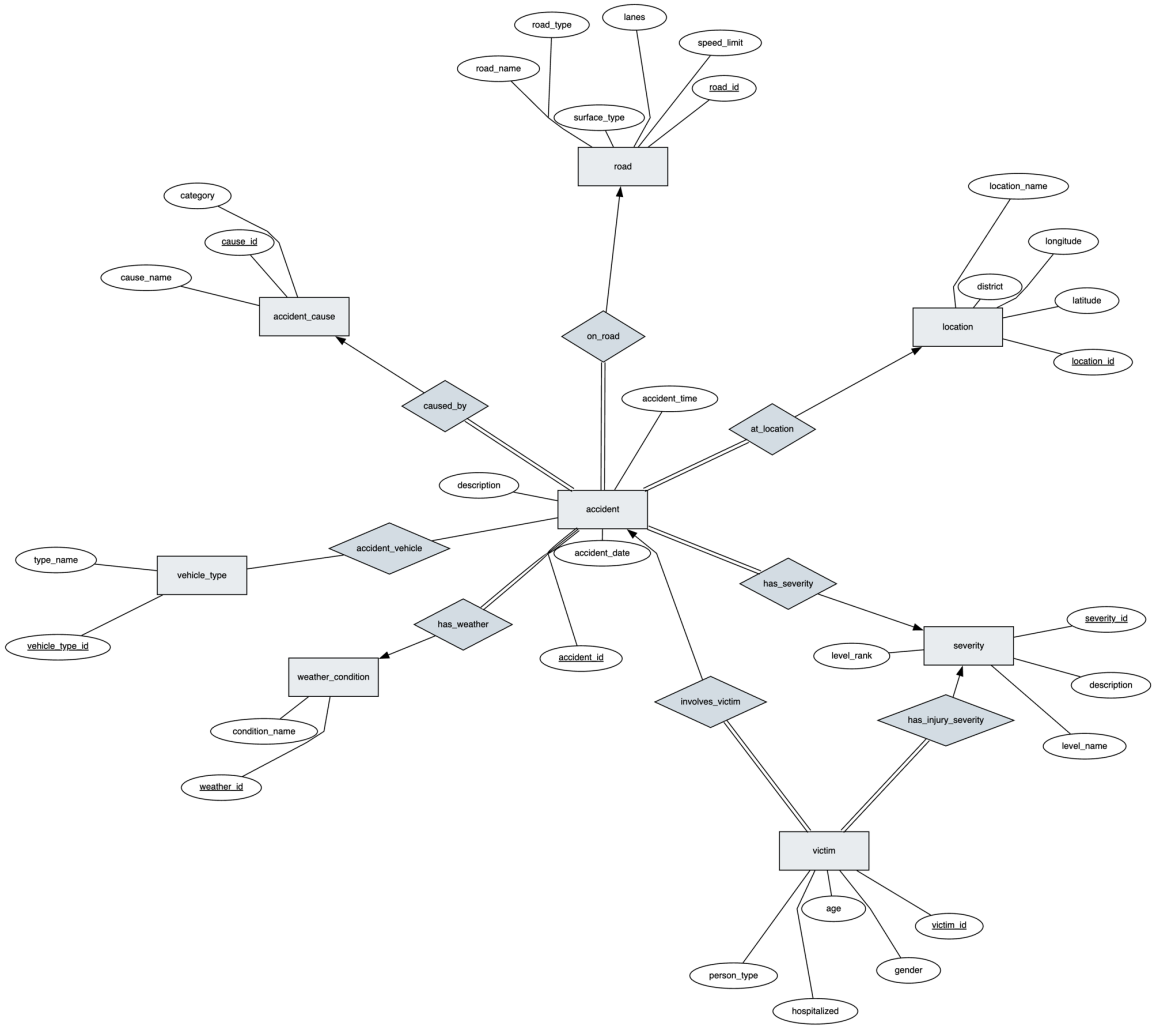
Entity	Attributes
accident	**accident_id**, accident_date, accident_time, description
location	**location_id**, location_name, district, latitude, longitude
road	**road_id**, road_name, road_type, lanes, surface_type, speed_limit
weather_condition	**weather_id**, condition_name

Entity	Attributes
severity	**severity_id**, level_name, level_rank, description
accident_cause	**cause_id**, cause_name, category
vehicle_type	**vehicle_type_id**, type_name
victim	**victim_id**, age, gender, person_type, hospitalized

Relationships and constraints.

Relationship	Entities	Cardinality	Participation
at_location	accident to location	N:1	accident total
on_road	accident to road	N:1	accident total
has_weather	accident to weather_condition	N:1	accident total
has_severity	accident to severity	N:1	accident total
caused_by	accident to accident_cause	N:1	accident total
accident_vehicle	accident to vehicle_type	M:N	each accident has at least one vehicle
involves_victim	accident to victim	1:N	victim total (a victim belongs to exactly one accident)
has_injury_severity	victim to severity	N:1	victim total

The ER diagram below shows these entities and relationships. Arrowheads point to the "one" side and double lines mark total participation.



6. Relational Schemas Obtained from the ER Diagram

Mapping rules applied.

- Each strong entity becomes a table with its key as the primary key.
- Each N:1 relationship is merged into the "many" side as a foreign key. accident therefore receives location_id, road_id, weather_id, severity_id and cause_id.
- The M:N relationship accident_vehicle becomes its own table, with a composite primary key of the two foreign keys.
- The 1:N relationship involves_victim puts accident_id in victim. The relationship has_injury_severity adds injury_severity_id to victim.

Primary keys are in **bold**. Foreign keys are marked FK.

#	Relation	Attributes
1	location	(**location_id**, location_name, district, latitude, longitude)
2	road	(**road_id**, road_name, road_type, lanes, surface_type,

#	Relation	Attributes
		speed_limit)
3	weather_condition	(**weather_id**, condition_name)
4	severity	(**severity_id**, level_name, level_rank, description)
5	accident_cause	(**cause_id**, cause_name, category)
6	vehicle_type	(**vehicle_type_id**, type_name)
7	accident	(**accident_id**, accident_date, accident_time, description, location_id FK, road_id FK, weather_id FK, severity_id FK, cause_id FK)
8	accident_vehicle	(**accident_id** FK, **vehicle_type_id** FK)
9	victim	(**victim_id**, accident_id FK, age, gender, person_type, hospitalized, injury_severity_id FK)

Foreign keys.

Child table.column	References	On update	On delete
accident.location_id	location.location_id	CASCADE	RESTRICT
accident.road_id	road.road_id	CASCADE	RESTRICT
accident.weather_id	weather_condition.weather_id	CASCADE	RESTRICT
accident.severity_id	severity.severity_id	CASCADE	RESTRICT
accident.cause_id	accident_cause.cause_id	CASCADE	RESTRICT
accident_vehicle.accident_id	accident.accident_id	CASCADE	CASCADE
accident_vehicle.vehicle_type_id	vehicle_type.vehicle_type_id	CASCADE	RESTRICT
victim.accident_id	accident.accident_id	CASCADE	CASCADE
victim.injury_severity_id	severity.severity_id	CASCADE	RESTRICT

Other constraints. UNIQUE on condition_name, level_name, level_rank, cause_name and type_name. CHECK: latitude in [-90, 90], longitude in [-180, 180], road_type in (Highway, Urban, Rural), lanes > 0, speed_limit > 0, level_rank in (1, 2, 3), age between 0 and 125, gender in (Male, Female, Other), person_type in (Driver, Passenger, Pedestrian). Indexes: accident(location_id), accident(accident_date) and accident(severity_id).

7. Functional Dependencies on Each Table

The dependencies below are those that must hold on each table. They follow from the keys and the business rules of the schema. Every attribute is functionally determined by the primary key, and any UNIQUE column is an additional candidate key.

Table	Functional dependencies
location	location_id → location_name, district, latitude, longitude
road	road_id → road_name, road_type, lanes, surface_type, speed_limit
weather_condition	weather_id → condition_name; condition_name → weather_id
severity	severity_id → level_name, level_rank, description; level_name → severity_id, level_rank, description; level_rank → severity_id, level_name, description
accident_cause	cause_id → cause_name, category; cause_name → cause_id, category
vehicle_type	vehicle_type_id → type_name; type_name → vehicle_type_id
accident	accident_id → accident_date, accident_time, description, location_id, road_id, weather_id, severity_id, cause_id
accident_vehicle	(accident_id, vehicle_type_id) → (no non-key attributes; the whole relation is the key)
victim	victim_id → accident_id, age, gender, person_type, hospitalized, injury_severity_id

Dependencies that deliberately do not hold. Showing these explains why the schema needs no further decomposition.

- `road_type` does not determine `speed_limit` or `lanes`. The sample data has Highways with limits of 100, 90 and 80.
- `district` is not determined by latitude or longitude in the schema, and `location_name` is not declared unique, so no non-key attribute determines another.
- `hospitalized` does not determine `injury_severity_id`, and `age` does not determine `person_type`.
- Road, weather, cause and severity details are not repeated in `accident`. They are reached through foreign keys, so `accident` has no dependency such as `road_id` → `road_type` inside it.

8. Normalization up to 3NF

The three conditions tested are:

- **1NF:** all attribute values are atomic, there are no repeating groups, and a primary key exists.
- **2NF:** 1NF plus no partial dependency of a non-key attribute on part of a composite key.
- **3NF:** 2NF plus no transitive dependency. For every FD X → A, either X is a superkey or A is a prime attribute.

Table	1NF	2NF	3NF
location	Atomic. Latitude and longitude are stored as two separate numbers.	Single-attribute key, so no partial dependency.	The only determinant is `location_id`, a superkey. Satisfied.

Table	1NF	2NF	3NF
road	Atomic.	Single-attribute key.	The only determinant is road_id. Satisfied.
weather_condition	Atomic.	Single-attribute key.	The determinants weather_id and condition_name are both candidate keys. Satisfied (also BCNF).
severity	Atomic.	Single-attribute key.	severity_id, level_name and level_rank are all candidate keys. Satisfied (also BCNF).
accident_cause	Atomic.	Single-attribute key.	cause_name → category has a candidate key on the left, so it is not transitive. Satisfied.
vehicle_type	Atomic.	Single-attribute key.	Both determinants are candidate keys. Satisfied.
accident	Atomic. The multi-valued vehicle list is moved out to accident_vehicle.	Single-attribute key.	No non-key attribute determines another. Road, weather, severity, cause and location attributes live in their own tables. Satisfied.
accident_vehicle	Atomic. Replaces a comma-separated vehicle list.	All attributes are prime, so no partial dependency is possible.	No non-prime attributes. Satisfied (also BCNF).
victim	Atomic. Repeating victims are separate rows, not columns.	Single-attribute key.	Only victim_id determines the other attributes. Satisfied.

Decomposition story. A single flat table accident_flat(accident_id, date, time, location_name, district, latitude, longitude, road_name, road_type, lanes, surface_type, speed_limit, weather, severity_name, severity_rank, cause_name, cause_category, vehicles, victims...) would violate each normal form:

- **1NF:** the lists of vehicles and victims are multi-valued.
- **2NF/3NF:** the transitive dependencies location_id → district, latitude, longitude, road_id → road_type, lanes, speed_limit, severity_name → severity_rank and cause_name → cause_category cause update anomalies. Changing a road's speed limit would require editing many accident rows, and a location that has no accident could not be stored at all.

Splitting these facts into the nine relations above removes the anomalies. Every table is in 3NF, and all of them are in BCNF as well, because every determinant is a candidate key.

9. Conclusion

The project turns an informal accident log into a clean, normalized relational design. The ER model maps to nine relations with explicit keys, referential actions and CHECK constraints. Each relation was checked

against its functional dependencies and is in 3NF (and BCNF), so the design is free of redundancy and update, insertion and deletion anomalies.

On top of the schema, the system provides a hotspot-ranking view, aggregate analytics, a history-based severity predictor in a stored procedure and an ACID transaction for recording incidents. Together these show how good schema design supports real analysis. Future work could add authentication, real police data, map visualisation, a richer model with per-vehicle and per-driver details, and a trained machine-learning predictor alongside the current frequency-based one.